Zela+ — Automação ESP32 (Sensor de Presença)

Documentação técnica do módulo de hardware que alimenta a detecção de presença/movimento do Zela+. Firmware em `firmware/zela_presenca/zela_presenca.ino`, recebido pelo backend em `POST /ingest/esp32` (`zela/api/ingestao.py`).

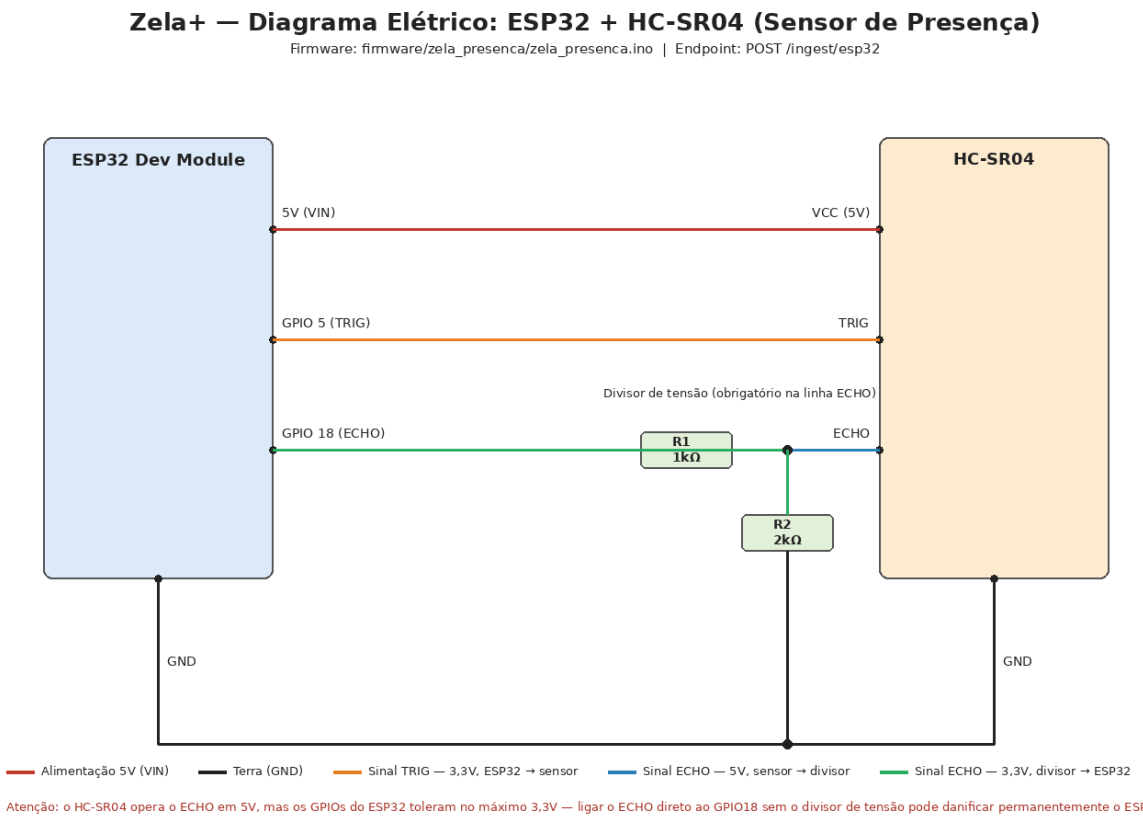
1. Visão geral

Um ESP32 com um sensor ultrassônico HC-SR04 monitora continuamente se há alguém no cômodo (presença detectada quando um objeto/pessoa está a até 150cm do sensor). O firmware só envia uma leitura quando o estado muda (presente ↔ ausente), evitando tráfego desnecessário. Essa leitura alimenta a regra de risco do sistema: se não houver nenhuma presença registrada nas últimas 6 horas, o Zela+ classifica o evento como `RISCO` (`zela/domain/monitoramento.py`) e aciona a escada de escalonamento de alerta para a família.

2. Lista de materiais (BOM)

Qtd.	Componente	Observação
1	ESP32 Dev Module (ex.: ESP32-WROOM-32 DevKit)	com WiFi integrado
1	Sensor ultrassônico HC-SR04	alcance ~2cm–400cm
1	Resistor 1kΩ (R1)	divisor de tensão da linha ECHO
1	Resistor 2kΩ (R2)	pode usar 2× 1kΩ em série se não tiver 2kΩ à mão
—	Jumpers macho-fêmea + protoboard	para montagem sem solda
1	Cabo USB (alimentação/gravação)	conforme o conector da placa (micro-USB ou USB-C)

3. Diagrama elétrico



4. Tabela de pinagem

HC-SR04	ESP32	Tipo de sinal	Observação
VCC	5V (VIN)	Alimentação	
GND	GND	Terra	Terra comum entre os dois componentes e o divisor
TRIG	GPIO 5	Saída digital 3,3V	Ligação direta, sem divisor
ECHO	GPIO 18 (via R1/R2)	Entrada digital	Precisa do divisor de tensão — ver seção 5

5. Por que o divisor de tensão

O HC-SR04 opera a linha `ECHO` em 5V, mas os GPIOs do ESP32 toleram no máximo ~3,3V (ultrapassar isso pode danificar a placa permanentemente). O divisor resistivo R1 (1kΩ) + R2 (2kΩ) entre `ECHO` e `GPIO 18` reduz a tensão para um nível seguro:

```
Vout = 5V × R2 / (R1 + R2) = 5V × 2kΩ / 3kΩ ≈ 3,33V
```

A linha `TRIG` não precisa de divisor porque é o ESP32 (3,3V) enviando o pulso para o sensor, não o contrário.

6. Lógica do firmware

Resumo de `firmware/zela_presenca/zela_presenca.ino`:

1. No boot: conecta ao WiFi (timeout de 30s) e sincroniza o relógio via NTP (`pool.ntp.org`) — o ESP32 não tem RTC com bateria própria, então sem isso os timestamps enviados seriam inválidos (época Unix).
2. A cada 2 segundos (`INTERVALO_LEITURA_MS`), dispara um pulso no `TRIG` e mede o tempo de retorno no `ECHO` para calcular a distância.
3. Considera "presença" quando a distância é > 0 e ≤ 150cm (`DISTANCIA_LIMIAR_CM`).
4. Só envia ao backend quando o estado muda (presente↔ausente) — não a cada leitura.
5. Se o WiFi cair, tenta reconectar a cada 15s (`INTERVALO_RECONEXAO_MS`) sem bloquear o loop principal.

7. Configuração de rede e segurança

Antes de gravar o firmware, preencher em `zela_presenca.ino`:

Constante	Valor
`WIFI_SSID` / `WIFI_SENHA`	credenciais da rede WiFi local
`URL_INGESTAO`	endereço do backend — rede local (`http://192.168.x.x:8000/ingest/esp32`) ou VPS pública (`https://seu-dominio.com/ingest/esp32`)
`TOKEN_INGESTAO`	mesmo valor de `ZELA_INGESTAO_TOKEN` no backend — obrigatório se o backend estiver exposto fora da rede local (ver `README.md`, seção de configuração)

Quando `TOKEN_INGESTAO` está preenchido, o firmware envia o header `X-Api-Key` em toda requisição; o backend rejeita com `401` qualquer requisição sem o token correto.

8. Fluxo de dados no sistema

```
ESP32 (mudança de estado) -> POST /ingest/esp32 (X-Api-Key)
 -> LeituraSensor(fonte=ESP32, tipo=PRESENCA) salva no SQLite
 -> job periódico (a cada 15 min, zela/api/scheduler.py)
    -> classificar_por_regra(): >=6h sem presença? -> StatusMonitoramento.RISCO
       -> aplicar_escalonamento() -> alerta enviado à família via WhatsApp
```

9. Checklist de montagem e teste

1. Montar o circuito conforme o diagrama da seção 3.

2. Abrir `firmware/zela_presenca/zela_presenca.ino` no Arduino IDE, placa "ESP32 Dev Module", preencher as constantes da seção 7.
3. Gravar e abrir o Serial Monitor (115200 bps).
4. Confirmar no log: conexão WiFi, sincronização NTP.
5. Mover a mão na frente do sensor (cruzando o limiar de 150cm) e confirmar `POST /ingest/esp32 -> 200` no Serial Monitor.
6. Confirmar que a leitura chegou no backend (painel Streamlit ou consulta direta ao `zela.db`).